Introduction to OOP, Classes, Objects, Methods, Attributes, Access Specifiers, Inheritance: Base Classes and Subclasses, Polymorphism

Introduction

Object-Oriented Programming (OOP) is a programming paradigm that organizes software design around objects rather than functions and logic. OOP models real-world entities as objects that contain both data (attributes) and behavior (methods). This approach makes code more modular, reusable, and easier to maintain.

Python is a multi-paradigm language with strong OOP support. Understanding OOP concepts is essential for building scalable applications, working with frameworks, and writing professional Python code. This comprehensive guide covers the fundamental OOP concepts: classes, objects, methods, attributes, access specifiers, inheritance, and polymorphism.

Topic 1: Introduction to Object-Oriented Programming

1.1 What is Object-Oriented Programming?

Object-Oriented Programming is a methodology for organizing programs around objects that bundle data and behavior together. Instead of focusing on functions that process data, OOP focuses on creating objects that contain both data and the methods to manipulate that data.

Core principles of OOP:

- **Encapsulation** - Bundling data and methods that operate on that data within a single unit (class)
- **Abstraction** - Hiding complex implementation details and showing only necessary features
- **Inheritance** - Creating new classes based on existing classes to promote code reuse
- **Polymorphism** - Using a unified interface for different data types or classes

1.2 Why Use OOP?

OOP provides several advantages over procedural programming:

Code Organization: Classes provide logical structure by grouping related data and functions together.

Reusability: Once a class is created, it can be instantiated multiple times and inherited by other classes.

Maintainability: Changes to a class automatically propagate to all its instances, reducing code duplication.

Real-World Modeling: OOP naturally maps to real-world entities, making programs more intuitive.

Scalability: OOP principles facilitate building large, complex applications through modular design.

1.3 OOP vs Procedural Programming

Procedural Programming:

```
def calculate_area(length, width):
return length * width
```

```
def calculate_perimeter(length, width):
return 2 * (length + width)
```

Data and functions are separate

```
rect_length = 5
rect_width = 3
area = calculate_area(rect_length, rect_width)
perimeter = calculate_perimeter(rect_length, rect_width)
```

Object-Oriented Programming:

```
class Rectangle:
def init(self, length, width):
self.length = length
self.width = width

def calculate_area(self):
    return self.length * self.width

def calculate_perimeter(self):
    return 2 * (self.length + self.width)
```

Data and methods are bundled together

```
rect = Rectangle(5, 3)
area = rect.calculate_area()
perimeter = rect.calculate_perimeter()
```

The OOP approach bundles related data and behavior, making code more organized and maintainable.

Topic 2: Classes and Objects

2.1 Understanding Classes

A class is a blueprint or template for creating objects. It defines the structure (attributes) and behavior (methods) that objects of that class will have.

Class syntax:

```
class ClassName:
"""Class docstring describing the class"""

# Class attributes (shared by all instances)
class_attribute = value

def __init__(self, parameters):
    """Constructor method"""
    # Instance attributes (unique to each instance)
    self.instance_attribute = value
```

```
def method_name(self, parameters):
    """Instance method"""
    # Method body
    pass
```

Simple class example:

```
class Dog:
"""A class representing a dog"""

# Class attribute
species = "Canis familiaris"

def __init__(self, name, age):
    """Initialize a new dog"""
    self.name = name  # Instance attribute
    self.age = age    # Instance attribute

def bark(self):
    """Make the dog bark"""
    print(f"{self.name} says Woof!")

def get_info(self):
    """Return dog information"""
    return f"{self.name} is {self.age} years old"
```

2.2 Creating Objects (Instances)

An object is a concrete instance of a class. Creating an object is called instantiation.

Creating objects:

Create dog objects

```
buddy = Dog("Buddy", 4)
miles = Dog("Miles", 9)
```

Access attributes

```
print(buddy.name) # Buddy
print(miles.age) # 9
print(buddy.species) # Canis familiaris (class attribute)
```

Call methods

```
buddy.bark() # Buddy says Woof!
print(miles.get_info()) # Miles is 9 years old
```

Multiple instances:

```
dogs = [
```

```
Dog("Max", 3),
Dog("Luna", 5),
Dog("Charlie", 2)
]

for dog in dogs:
print(dog.get_info())
```

Each object is independent with its own attribute values, but all share the same methods defined in the class.

2.3 The __init__ Method (Constructor)

The __init__ method is a special method called automatically when an object is created. It initializes the object's attributes.

Constructor with validation:

```
class BankAccount:
def init(self, account_holder, initial_balance=0):
"""Initialize a bank account"""
self.account_holder = account_holder

    if initial_balance < 0:
        raise ValueError("Initial balance cannot be negative")
    self.balance = initial_balance
    self.transaction_count = 0

def get_balance(self):
    return self.balance
```

Create accounts

```
account1 = BankAccount("Alice", 1000)
account2 = BankAccount("Bob") # Uses default balance of 0
```

The self parameter:

The first parameter of every instance method is self, which represents the instance calling the method. Python automatically passes the instance as the first argument.

```
class Example:
def init(self, value):
self.value = value # self refers to the current instance

def display(self):
    print(self.value)   # Access instance attribute via self

obj = Example(42)
obj.display() # Python automatically passes obj as self
```

2.4 Class vs Instance Attributes

Class attributes are shared by all instances of the class. **Instance attributes** are unique to each instance.

```
class Car:
# Class attribute (shared)
wheels = 4

def __init__(self, brand, model):
    # Instance attributes (unique to each instance)
    self.brand = brand
    self.model = model

car1 = Car("Toyota", "Camry")
car2 = Car("Honda", "Civic")

print(car1.brand) # Toyota (instance attribute)
print(car2.brand) # Honda (instance attribute)
print(car1.wheels) # 4 (class attribute, same for all)
print(car2.wheels) # 4 (class attribute, same for all)
```

Modify class attribute affects all instances

```
Car.wheels = 6
print(car1.wheels) # 6
print(car2.wheels) # 6
```

Topic 3: Methods

3.1 Instance Methods

Instance methods operate on instance data and can access both instance and class attributes. The first parameter is always self.

Instance methods example:

```
class Student:
def init(self, name, roll_number):
self.name = name
self.roll_number = roll_number
self.grades = []

def add_grade(self, grade):
    """Add a grade to the student's record"""
    if 0 <= grade <= 100:
        self.grades.append(grade)
    else:
        print("Invalid grade")

def calculate_average(self):
    """Calculate average grade"""
```

```
    if not self.grades:
        return 0
    return sum(self.grades) / len(self.grades)

def display_info(self):
    """Display student information"""
    avg = self.calculate_average()
    print(f"Name: {self.name}")
    print(f"Roll Number: {self.roll_number}")
    print(f"Average Grade: {avg:.2f}")
```

```
student = Student("Alice", "CS101")
student.add_grade(85)
student.add_grade(90)
student.add_grade(88)
student.display_info()
```

3.2 Class Methods

Class methods work with class attributes rather than instance attributes. They use the `@classmethod` decorator and take `cls` as the first parameter.

Class methods example:

```
class Employee:
company_name = "TechCorp"
employee_count = 0

def __init__(self, name, position):
    self.name = name
    self.position = position
    Employee.employee_count += 1

@classmethod
def get_employee_count(cls):
    """Return total number of employees"""
    return cls.employee_count

@classmethod
def change_company_name(cls, new_name):
    """Change company name for all employees"""
    cls.company_name = new_name
```

```
emp1 = Employee("Alice", "Developer")
emp2 = Employee("Bob", "Designer")

print(Employee.get_employee_count()) # 2
Employee.change_company_name("NewTech Inc")
print(Employee.company_name) # NewTech Inc
```

3.3 Static Methods

Static methods don't access instance or class data. They use the @staticmethod decorator and don't take self or cls as parameters.

Static methods example:

```
class MathOperations:
@staticmethod
def add(x, y):
"""Add two numbers"""
return x + y
```

```
@staticmethod
def is_even(number):
    """Check if a number is even"""
    return number % 2 == 0

@staticmethod
def factorial(n):
    """Calculate factorial"""
    if n == 0 or n == 1:
        return 1
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result
```

Call static methods without creating an instance

```
print(MathOperations.add(5, 3)) # 8
print(MathOperations.is_even(10)) # True
print(MathOperations.factorial(5)) # 120
```

3.4 Special Methods (Magic Methods)

Special methods (also called magic or dunder methods) have double underscores before and after their names. They provide special functionality.

Common special methods:

```
class Book:
def __init__(self, title, author, pages):
self.title = title
self.author = author
self.pages = pages
```

```
def __str__(self):
    """String representation for users"""
    return f"'{self.title}' by {self.author}"
```

```
def __repr__(self):
    """String representation for developers"""
    return f"Book('{self.title}', '{self.author}', {self.pages})"

def __len__(self):
    """Return length (number of pages)"""
    return self.pages

def __eq__(self, other):
    """Check equality based on title and author"""
    return self.title == other.title and self.author == other.author
```

```
book1 = Book("Python Programming", "John Doe", 350)
book2 = Book("Python Programming", "John Doe", 350)

print(book1) # 'Python Programming' by John Doe
print(len(book1)) # 350
print(book1 == book2) # True
```

Topic 4: Attributes

4.1 Instance Attributes

Instance attributes are variables that belong to specific instances and are typically defined in __init__.

Working with instance attributes:

```
class Person:
def init(self, name, age, city):
self.name = name
self.age = age
self.city = city
```

```
def update_age(self, new_age):
    """Update person's age"""
    if new_age > 0:
        self.age = new_age

def relocate(self, new_city):
    """Update person's city"""
    self.city = new_city
```

```
person = Person("Alice", 25, "Boston")
```

Access attributes

```
print(person.name) # Alice
```

Modify attributes

```
person.age = 26
person.relocate("New York")

print(f"{person.name} is {person.age} years old and lives in {person.city}")
```

4.2 Class Attributes

Class attributes are shared by all instances and are defined at the class level.

Class attributes example:

```
class Circle:
pi = 3.14159 # Class attribute

def __init__(self, radius):
    self.radius = radius  # Instance attribute

def area(self):
    return Circle.pi * self.radius ** 2

def circumference(self):
    return 2 * Circle.pi * self.radius

circle1 = Circle(5)
circle2 = Circle(10)

print(circle1.area()) # Uses shared pi value
print(circle2.area()) # Uses same pi value
```

4.3 Property Decorators

Properties allow controlled access to attributes through getter and setter methods.

Using @property decorator:

```
class Temperature:
def init(self, celsius):
self._celsius = celsius

@property
def celsius(self):
    """Get temperature in Celsius"""
    return self._celsius

@celsius.setter
def celsius(self, value):
    """Set temperature in Celsius"""
    if value < -273.15:
        raise ValueError("Temperature cannot be below absolute zero")
    self._celsius = value

@property
```

```
def fahrenheit(self):
    """Get temperature in Fahrenheit"""
    return (self._celsius * 9/5) + 32

@fahrenheit.setter
def fahrenheit(self, value):
    """Set temperature using Fahrenheit"""
    self._celsius = (value - 32) * 5/9
```

```
temp = Temperature(25)
print(temp.celsius) # 25
print(temp.fahrenheit) # 77.0

temp.fahrenheit = 86
print(temp.celsius) # 30.0
```

Topic 5: Access Specifiers

5.1 Understanding Access Levels in Python

Unlike languages like Java or C++, Python doesn't have strict access control keywords. Instead, it uses naming conventions to indicate intended access levels.

Three access levels:

- **Public** - No underscore prefix, accessible everywhere (default)
- **Protected** - Single underscore prefix (_), intended for internal use and subclasses
- **Private** - Double underscore prefix (__), name mangling applied to discourage external access

5.2 Public Attributes and Methods

Public members have no underscore prefix and are accessible from anywhere.

Public members example:

```
class BankAccount:
def init(self, account_holder):
self.account_holder = account_holder # Public attribute
self.account_number = self._generate_account_number()
```

```
def deposit(self, amount):  # Public method
    """Deposit money into account"""
    if amount > 0:
        print(f"Deposited ${amount}")
    else:
        print("Invalid amount")

def _generate_account_number(self):
    """Generate account number"""
    import random
```

```
    return random.randint(10000, 99999)
```

```
account = BankAccount("Alice")
print(account.account_holder) # Accessible (public)
account.deposit(500) # Accessible (public)
```

5.3 Protected Attributes and Methods

Protected members have a single underscore prefix. This is a convention indicating they're intended for internal use and subclasses, but they're still technically accessible.

Protected members example:

```
class Vehicle:
def init(self, brand, model):
self.brand = brand # Public
self._model = model # Protected (convention)
self._mileage = 0 # Protected
```

```
def _update_mileage(self, miles):  # Protected method
    """Internal method for updating mileage"""
    self._mileage += miles

def drive(self, miles):
    """Public method to drive the vehicle"""
    print(f"Driving {miles} miles")
    self._update_mileage(miles)  # Call protected method internally
```

```
car = Vehicle("Toyota", "Camry")
print(car.brand) # OK (public)
print(car._model) # Accessible but discouraged (protected)
car.drive(100) # OK (public)
```

car._update_mileage(50) # Accessible but not recommended

5.4 Private Attributes and Methods

Private members have a double underscore prefix. Python applies name mangling, changing `__attribute` to `_ClassName__attribute`, making direct access more difficult.

Private members example:

```
class BankAccount:
def init(self, account_holder, initial_balance):
self.account_holder = account_holder # Public
self.__balance = initial_balance # Private
```

```
def deposit(self, amount):
    """Deposit money"""
    if amount > 0:
        self.__balance += amount
```

```
        self.__log_transaction("deposit", amount)  # Private method

    def withdraw(self, amount):
        """Withdraw money"""
        if 0 < amount <= self.__balance:
            self.__balance -= amount
            self.__log_transaction("withdraw", amount)
        else:
            print("Insufficient funds")

    def get_balance(self):
        """Public method to access private balance"""
        return self.__balance

    def __log_transaction(self, type, amount):  # Private method
        """Log transaction (internal use only)"""
        print(f"Transaction: {type} ${amount}")
```

account = BankAccount("Alice", 1000)
account.deposit(500)
print(account.get_balance()) # 1500 (access via public method)

print(account.__balance) # AttributeError (private)

account.__log_transaction("test", 100) # AttributeError (private)

Name-mangled access (not recommended)

print(account._BankAccount__balance) # 1500 (works but discouraged)

5.5 Access Specifiers Comparison

Type	Syntax	Access Level	Use Case
Public	name	Everywhere	Normal attributes and methods
Protected	_name	Class and subclasses	Internal implementation details
Private	__name	Class only (name mangled)	Sensitive data, prevent conflicts

Table 1: Access specifier comparison in Python

Complete example:
class Employee:
def **init**(self, name, emp_id, salary):
self.name = name # Public - anyone can access
self._emp_id = emp_id # Protected - internal use
self.__salary = salary # Private - sensitive data

```
def get_salary(self):            # Public method to access private data
    return self.__salary

def _calculate_bonus(self):      # Protected - for internal use
    return self.__salary * 0.10

def __validate_salary(self):     # Private - internal validation
    return self.__salary > 0
```

```
emp = Employee("Alice", "E001", 50000)
print(emp.name) # OK (public)
print(emp._emp_id) # Accessible but discouraged (protected)
print(emp.get_salary()) # OK (public getter for private data)
```

print(emp.__salary) # Error (private)

Topic 6: Inheritance

6.1 Understanding Inheritance

Inheritance is a mechanism where a new class (subclass/derived class) is based on an existing class (superclass/base class). The subclass inherits attributes and methods from the superclass, promoting code reuse.

Benefits of inheritance:

- **Code Reusability** - Avoid duplicating code by inheriting from existing classes
- **Hierarchical Classification** - Model real-world relationships (IS-A relationship)
- **Extensibility** - Add new features to existing code without modification
- **Maintainability** - Changes to base class automatically propagate to subclasses

Inheritance Hierarchy Diagram:

```
    Animal
   /     \
  Dog     Cat
 /   \
```

Poodle Bulldog

6.2 Creating Subclasses

A subclass is created by specifying the parent class in parentheses after the class name.

Basic inheritance syntax:

```
class ParentClass:
def parent_method(self):
print("This is the parent class method")
```

```
class ChildClass(ParentClass):
def child_method(self):
print("This is the child class method")
```

Child inherits from parent

```
child = ChildClass()
child.parent_method() # Inherited method
child.child_method() # Own method
```

Practical inheritance example:

```
class Person:
def init(self, name, age):
self.name = name
self.age = age

def introduce(self):
    print(f"My name is {self.name} and I am {self.age} years old")

class Student(Person):
def init(self, name, age, student_id):
super().init(name, age) # Call parent constructor
self.student_id = student_id
self.grades = []

def add_grade(self, grade):
    self.grades.append(grade)

def introduce(self):
    super().introduce()  # Call parent method
    print(f"My student ID is {self.student_id}")

student = Student("Alice", 20, "S12345")
student.introduce()
student.add_grade(85)
```

6.3 The super() Function

The `super()` function provides access to methods from the parent class. It's commonly used to call the parent's `__init__` method.

Using super():

```
class Animal:
def init(self, name, species):
self.name = name
self.species = species
```

```
def make_sound(self):
    print(f"{self.name} makes a sound")
```

```
class Dog(Animal):
def init(self, name, breed):
super().init(name, "Dog") # Call parent constructor
self.breed = breed
```

```
def make_sound(self):
    super().make_sound()  # Call parent method
    print(f"{self.name} barks")
```

```
dog = Dog("Buddy", "Golden Retriever")
dog.make_sound()
```

Output:

Buddy makes a sound

Buddy barks

6.4 Method Overriding

Subclasses can override parent methods to provide specialized behavior.

Method overriding example:

```
class Shape:
def init(self, name):
self.name = name
```

```
def area(self):
    return 0

def describe(self):
    print(f"This is a {self.name}")
```

```
class Rectangle(Shape):
def init(self, width, height):
super().init("Rectangle")
self.width = width
self.height = height
```

```
def area(self):  # Override parent method
    return self.width * self.height
```

```
class Circle(Shape):
def init(self, radius):
```

```
super().init("Circle")
self.radius = radius

def area(self):  # Override parent method
    return 3.14159 * self.radius ** 2

rect = Rectangle(5, 3)
circle = Circle(4)

print(f"Rectangle area: {rect.area()}") # 15
print(f"Circle area: {circle.area():.2f}") # 50.27
```

6.5 Multiple Inheritance

Python supports multiple inheritance, where a class can inherit from multiple parent classes.

Multiple inheritance example:

```
class Flyer:
def fly(self):
print("Flying in the air")

class Swimmer:
def swim(self):
print("Swimming in water")

class Duck(Flyer, Swimmer):
def quack(self):
print("Quack quack!")

duck = Duck()
duck.fly() # From Flyer
duck.swim() # From Swimmer
duck.quack() # Own method
```

Method Resolution Order (MRO):

When multiple inheritance is used, Python follows the Method Resolution Order to determine which method to call.

```
class A:
def method(self):
print("Method from A")

class B(A):
def method(self):
print("Method from B")

class C(A):
def method(self):
print("Method from C")
```

```
class D(B, C):
pass

d = D()
d.method() # Method from B (follows MRO)
print(D.mro()) # Show method resolution order
```

6.6 Types of Inheritance

Single Inheritance:

```
class Parent:
pass

class Child(Parent):
pass
```

Multi-level Inheritance:

```
class Grandparent:
pass

class Parent(Grandparent):
pass

class Child(Parent):
pass
```

Hierarchical Inheritance:

```
class Parent:
pass

class Child1(Parent):
pass

class Child2(Parent):
pass
```

Inheritance Types Diagram:

```
Single: Multi-level: Hierarchical: Multiple:
Parent Grandparent Parent Parent1 Parent2
| | / \ \ /
Child Parent Child1 Child2 Child
|
Child
```

Complete inheritance example:

```
class Vehicle:
def init(self, brand, model, year):
self.brand = brand
self.model = model
self.year = year
```

```
def display_info(self):
    print(f"{self.year} {self.brand} {self.model}")
```

```
class Car(Vehicle):
def init(self, brand, model, year, doors):
super().init(brand, model, year)
self.doors = doors
```

```
def display_info(self):
    super().display_info()
    print(f"Doors: {self.doors}")
```

```
class ElectricCar(Car):
def init(self, brand, model, year, doors, battery_capacity):
super().init(brand, model, year, doors)
self.battery_capacity = battery_capacity
```

```
def display_info(self):
    super().display_info()
    print(f"Battery: {self.battery_capacity} kWh")
```

```
tesla = ElectricCar("Tesla", "Model 3", 2024, 4, 75)
tesla.display_info()
```

Topic 7: Polymorphism

7.1 Understanding Polymorphism

Polymorphism means "many forms" and refers to the ability to use a common interface for different data types or classes. In Python, polymorphism allows objects of different classes to be used interchangeably if they implement the same methods.

Types of polymorphism:

- **Duck Typing** - "If it walks like a duck and quacks like a duck, it's a duck"
- **Method Overriding** - Subclasses provide specific implementations of parent methods
- **Operator Overloading** - Defining how operators work with custom objects

Polymorphism Concept Diagram:

```
Common Interface: speak()
|
|---> Dog.speak() → "Woof!"
|---> Cat.speak() → "Meow!"
|---> Duck.speak() → "Quack!"
```

Same method name, different behaviors

7.2 Duck Typing Polymorphism

Python uses duck typing, where the type or class of an object is less important than the methods it defines.

Duck typing example:

```
class Dog:
def speak(self):
return "Woof!"

def move(self):
    return "Running on four legs"

class Cat:
def speak(self):
return "Meow!"

def move(self):
    return "Walking silently"

class Duck:
def speak(self):
return "Quack!"

def move(self):
    return "Waddling"

def animal_sounds(animal):
"""Function works with any object that has speak() method"""
print(animal.speak())
print(animal.move())
```

All three work with the same function

```
dog = Dog()
cat = Cat()
duck = Duck()

animal_sounds(dog)
animal_sounds(cat)
animal_sounds(duck)
```

7.3 Polymorphism Through Inheritance

Subclasses can override parent methods to provide specialized behavior while maintaining a common interface.

Polymorphism with inheritance:

```
class Shape:
def init(self, name):
self.name = name
```

```
def area(self):
    raise NotImplementedError("Subclass must implement area()")

def perimeter(self):
    raise NotImplementedError("Subclass must implement perimeter()")
```

class Rectangle(Shape):
def **init**(self, width, height):
super().**init**("Rectangle")
self.width = width
self.height = height

```
def area(self):
    return self.width * self.height

def perimeter(self):
    return 2 * (self.width + self.height)
```

class Circle(Shape):
def **init**(self, radius):
super().**init**("Circle")
self.radius = radius

```
def area(self):
    return 3.14159 * self.radius ** 2

def perimeter(self):
    return 2 * 3.14159 * self.radius
```

class Triangle(Shape):
def **init**(self, a, b, c):
super().**init**("Triangle")
self.a = a
self.b = b
self.c = c

```
def area(self):
    s = (self.a + self.b + self.c) / 2
    return (s * (s - self.a) * (s - self.b) * (s - self.c)) ** 0.5

def perimeter(self):
    return self.a + self.b + self.c
```

Polymorphic behavior - same interface, different implementations

```
shapes = [
Rectangle(5, 3),
Circle(4),
Triangle(3, 4, 5)
]

for shape in shapes:
print(f"{shape.name}:")
print(f" Area: {shape.area():.2f}")
print(f" Perimeter: {shape.perimeter():.2f}")
```

7.4 Operator Overloading

Operator overloading allows custom classes to define behavior for built-in operators using special methods.

Operator overloading example:

```
class Vector:
def init(self, x, y):
self.x = x
self.y = y

def __add__(self, other):
    """Overload + operator"""
    return Vector(self.x + other.x, self.y + other.y)

def __sub__(self, other):
    """Overload - operator"""
    return Vector(self.x - other.x, self.y - other.y)

def __mul__(self, scalar):
    """Overload * operator for scalar multiplication"""
    return Vector(self.x * scalar, self.y * scalar)

def __eq__(self, other):
    """Overload == operator"""
    return self.x == other.x and self.y == other.y

def __str__(self):
    """String representation"""
    return f"Vector({self.x}, {self.y})"

v1 = Vector(2, 3)
v2 = Vector(4, 5)

v3 = v1 + v2 # Uses add
v4 = v2 - v1 # Uses sub
v5 = v1 * 3 # Uses mul
```

```
print(v3) # Vector(6, 8)
print(v4) # Vector(2, 2)
print(v5) # Vector(6, 9)
print(v1 == v2) # False
```

Common operator overloading methods:

Method	Operator
__add__(self, other)	+
__sub__(self, other)	-
__mul__(self, other)	*
__truediv__(self, other)	/
__eq__(self, other)	==
__lt__(self, other)	<
__gt__(self, other)	>
__len__(self)	len()
__str__(self)	str()
__repr__(self)	repr()

Table 2: Common operator overloading methods

7.5 Polymorphism in Practice

Real-world polymorphism example:

```
class PaymentMethod:
def pay(self, amount):
raise NotImplementedError("Subclass must implement pay()")

class CreditCard(PaymentMethod):
def init(self, card_number):
self.card_number = card_number

def pay(self, amount):
    print(f"Paid ${amount} using Credit Card ending in {self.card_number[-4:]}")

class PayPal(PaymentMethod):
def init(self, email):
self.email = email

def pay(self, amount):
    print(f"Paid ${amount} using PayPal account {self.email}")
```

```
class BankTransfer(PaymentMethod):
def init(self, account_number):
self.account_number = account_number

def pay(self, amount):
    print(f"Paid ${amount} via Bank Transfer from account {self.account_number}")

def process_payment(payment_method, amount):
"""Process payment using any payment method"""
payment_method.pay(amount)
```

Different payment methods, same interface

```
credit_card = CreditCard("1234567890123456")
paypal = PayPal("user@example.com")
bank = BankTransfer("9876543210")

process_payment(credit_card, 100)
process_payment(paypal, 75)
process_payment(bank, 200)
```

Best Practices

1. **Use meaningful class names** - Class names should be nouns in PascalCase (e.g., BankAccount, StudentRecord)
2. **Follow single responsibility** - Each class should have one clear purpose
3. **Favor composition over inheritance** - Use inheritance for IS-A relationships, composition for HAS-A
4. **Use properties for controlled access** - Implement getters/setters using @property decorator
5. **Document classes and methods** - Use docstrings to explain purpose and usage
6. **Keep classes focused** - Avoid "god classes" that do too much
7. **Use access conventions appropriately** - Public for interface, protected for internal, private for sensitive
8. **Implement special methods** - Define __str__, __repr__, __eq__ for better usability
9. **Validate in constructors** - Check input validity in __init__ to maintain object integrity
10. **Use type hints** - Document expected types for attributes and method parameters

Common Pitfalls to Avoid

- **Mutable class attributes** - Avoid using mutable objects as class attributes (use instance attributes instead)
- **Forgetting self parameter** - All instance methods must have self as first parameter
- **Not calling super().__init__()** - Always call parent constructor when overriding __init__
- **Overusing inheritance** - Don't create deep inheritance hierarchies; prefer composition
- **Breaking encapsulation** - Don't access private attributes directly; use public methods

- **Circular imports** - Avoid mutual dependencies between classes in different modules
- **Not implementing __repr__** - Implement for better debugging experience
- **Ignoring SOLID principles** - Follow Single Responsibility, Open/Closed, and other OOP principles
- **Multiple inheritance complexity** - Use multiple inheritance sparingly due to MRO complexity
- **Not handling exceptions** - Validate inputs and handle errors appropriately in methods

Practice Problems

Section A: Classes and Objects Basics

Problem 1: Create a Rectangle class with attributes length and width. Include methods to calculate area and perimeter.

Problem 2: Design a BankAccount class with attributes for account holder name and balance. Implement methods for deposit, withdraw, and check balance.

Problem 3: Create a Book class with title, author, and pages attributes. Implement __str__ and __repr__ methods.

Problem 4: Design a Student class with name, roll number, and a list of grades. Include methods to add grades and calculate average.

Problem 5: Create a Circle class with radius as attribute. Implement methods to calculate area, circumference, and diameter.

Problem 6: Design a Product class for an e-commerce system with name, price, and quantity attributes. Include a method to calculate total value.

Section B: Methods and Attributes

Problem 7: Create a Counter class with a class attribute to track total instances created. Use instance methods to increment and decrement a counter value.

Problem 8: Design a Temperature class with celsius attribute and property decorators to get/set temperature in both Celsius and Fahrenheit.

Problem 9: Create a Person class with static methods to validate email addresses and phone numbers.

Problem 10: Design a Library class with class methods to track total books and instances created.

Problem 11: Create a Date class with day, month, year attributes. Implement comparison operators (<, >, ==).

Problem 12: Design a Time class with hour, minute, second. Implement __add__ to add two time objects.

Section C: Access Specifiers

Problem 13: Create a CreditCard class with public card holder name, protected card number (last 4 digits visible), and private CVV.

Problem 14: Design a User class with public username, protected email, and private password. Implement methods for password validation and reset.

Problem 15: Create an Employee class demonstrating all three access levels (public, protected, private) with appropriate attributes and methods.

Problem 16: Design a Database class with private connection details and public methods to query data.

Section D: Inheritance

Problem 17: Create a Vehicle base class with brand and model. Create Car and Motorcycle subclasses with specific attributes.

Problem 18: Design a Shape base class with abstract area() and perimeter() methods. Create Rectangle, Circle, and Triangle subclasses.

Problem 19: Create an Animal base class with make_sound() method. Create Dog, Cat, and Bird subclasses that override this method.

Problem 20: Design a Person base class. Create Student and Teacher subclasses with specific attributes and methods.

Problem 21: Create a multi-level inheritance hierarchy: LivingBeing → Animal → Mammal → Dog.

Problem 22: Design a hierarchical inheritance with Employee as base and Manager, Developer, and Designer as subclasses.

Problem 23: Create a BankAccount base class and SavingsAccount and CheckingAccount subclasses with different interest calculation methods.

Problem 24: Implement multiple inheritance: create Flyer and Swimmer classes, then a Duck class inheriting from both.

Section E: Polymorphism

Problem 25: Create different payment classes (CreditCard, DebitCard, UPI) with a common process_payment() method. Demonstrate polymorphism.

Problem 26: Design a media player system with Audio and Video classes that have a common play() method.

Problem 27: Create a Document base class with open() and save() methods. Create PDFDocument, WordDocument, and TextDocument subclasses.

Problem 28: Implement operator overloading for a Fraction class supporting +, -, *, / operations.

Problem 29: Create a Matrix class with operator overloading for addition and multiplication.

Problem 30: Design a Money class that supports addition, subtraction, and comparison of monetary values.

Section F: Advanced OOP Concepts

Problem 31: Create an abstract base class Database with abstract methods. Implement MySQLDatabase and PostgreSQLDatabase subclasses.

Problem 32: Design a singleton class that ensures only one instance is ever created.

Problem 33: Implement a Stack class with push, pop, peek, and is_empty methods using a list internally.

Problem 34: Create a Queue class implementing enqueue, dequeue, front, and is_empty methods.

Problem 35: Design a LinkedList class with methods to insert, delete, and search nodes.

Problem 36: Implement a BinarySearchTree class with insert, search, and traversal methods.

Section G: Real-World Applications

Problem 37: Create a complete library management system with Book, Member, and Library classes.

Problem 38: Design a university system with Student, Course, Professor, and Department classes.

Problem 39: Implement a simple e-commerce system with Product, Cart, Order, and Customer classes.

Problem 40: Create a hotel reservation system with Room, Guest, Reservation, and Hotel classes.

Problem 41: Design a restaurant order management system with MenuItem, Order, Customer, and Restaurant classes.

Problem 42: Implement a vehicle rental system with different vehicle types and rental calculations.

Section H: Design Patterns

Problem 43: Implement the Factory pattern to create different types of database connections.

Problem 44: Create an Observer pattern where multiple objects are notified when a subject changes.

Problem 45: Implement a Builder pattern for constructing complex objects step-by-step.

Problem 46: Design a Strategy pattern for different sorting algorithms.

Problem 47: Create a Decorator pattern to add functionality to objects dynamically.

Problem 48: Implement a Command pattern for undo/redo functionality.

Section I: Exception Handling in OOP

Problem 49: Create custom exception classes (InsufficientFundsException, InvalidAccountException) for a banking system.

Problem 50: Design a FileManager class with proper exception handling for file operations.

Solutions to Practice Problems

Section A Solutions: Classes and Objects Basics

Problem 1: Rectangle class

```
class Rectangle:
def init(self, length, width):
"""Initialize rectangle with length and width"""
self.length = length
self.width = width

def area(self):
    """Calculate area of rectangle"""
    return self.length * self.width

def perimeter(self):
    """Calculate perimeter of rectangle"""
    return 2 * (self.length + self.width)

def is_square(self):
    """Check if rectangle is a square"""
    return self.length == self.width

rect = Rectangle(5, 3)
print(f"Area: {rect.area()}") # 15
print(f"Perimeter: {rect.perimeter()}") # 16
print(f"Is square: {rect.is_square()}") # False
```

Problem 2: BankAccount class

```
class BankAccount:
def init(self, account_holder, initial_balance=0):
"""Initialize bank account"""
self.account_holder = account_holder
self.balance = initial_balance
self.transactions = []

def deposit(self, amount):
    """Deposit money into account"""
    if amount > 0:
        self.balance += amount
        self.transactions.append(f"Deposited: ${amount}")
        print(f"Deposited ${amount}. New balance: ${self.balance}")
    else:
        print("Invalid deposit amount")

def withdraw(self, amount):
    """Withdraw money from account"""
    if amount > self.balance:
```

```
        print("Insufficient funds")
    elif amount <= 0:
        print("Invalid withdrawal amount")
    else:
        self.balance -= amount
        self.transactions.append(f"Withdrew: ${amount}")
        print(f"Withdrew ${amount}. New balance: ${self.balance}")

def check_balance(self):
    """Display current balance"""
    print(f"Account holder: {self.account_holder}")
    print(f"Current balance: ${self.balance}")
    return self.balance
```

```
account = BankAccount("Alice", 1000)
account.deposit(500)
account.withdraw(200)
account.check_balance()
```

Problem 3: Book class

```
class Book:
def init(self, title, author, pages):
"""Initialize book with title, author, and pages"""
self.title = title
self.author = author
self.pages = pages
```

```
def __str__(self):
    """User-friendly string representation"""
    return f"'{self.title}' by {self.author}"

def __repr__(self):
    """Developer-friendly representation"""
    return f"Book(title='{self.title}', author='{self.author}', pages={self.pages})"

def __len__(self):
    """Return number of pages"""
    return self.pages
```

```
book = Book("Python Programming", "John Doe", 450)
print(str(book)) # 'Python Programming' by John Doe
print(repr(book)) # Book(title='Python Programming', author='John Doe', pages=450)
print(len(book)) # 450
```

Problem 4: Student class

```
class Student:
def init(self, name, roll_number):
```

```
"""Initialize student with name and roll number"""
self.name = name
self.roll_number = roll_number
self.grades = []

def add_grade(self, subject, grade):
    """Add a grade for a subject"""
    if 0 <= grade <= 100:
        self.grades.append({'subject': subject, 'grade': grade})
        print(f"Added grade: {subject} - {grade}")
    else:
        print("Invalid grade. Must be between 0 and 100")

def calculate_average(self):
    """Calculate average of all grades"""
    if not self.grades:
        return 0
    total = sum(g['grade'] for g in self.grades)
    return total / len(self.grades)

def display_report(self):
    """Display student report card"""
    print(f"\n--- Report Card ---")
    print(f"Name: {self.name}")
    print(f"Roll Number: {self.roll_number}")
    print("\nGrades:")
    for g in self.grades:
        print(f"  {g['subject']}: {g['grade']}")
    print(f"\nAverage: {self.calculate_average():.2f}")

student = Student("Alice", "CS101")
student.add_grade("Math", 85)
student.add_grade("Physics", 90)
student.add_grade("Chemistry", 88)
student.display_report()
```

Problem 5: Circle class

```
class Circle:
pi = 3.14159

def __init__(self, radius):
    """Initialize circle with radius"""
    self.radius = radius

def area(self):
    """Calculate area of circle"""
    return Circle.pi * self.radius ** 2
```

```
def circumference(self):
    """Calculate circumference of circle"""
    return 2 * Circle.pi * self.radius

def diameter(self):
    """Calculate diameter of circle"""
    return 2 * self.radius
```

```
circle = Circle(5)
print(f"Radius: {circle.radius}")
print(f"Area: {circle.area():.2f}")
print(f"Circumference: {circle.circumference():.2f}")
print(f"Diameter: {circle.diameter()}")
```

Problem 6: Product class

```
class Product:
def init(self, name, price, quantity):
"""Initialize product"""
self.name = name
self.price = price
self.quantity = quantity
```

```
def total_value(self):
    """Calculate total value of product in inventory"""
    return self.price * self.quantity

def update_quantity(self, amount):
    """Update product quantity"""
    self.quantity += amount
    if self.quantity < 0:
        self.quantity = 0

def apply_discount(self, percentage):
    """Apply discount to product price"""
    if 0 < percentage < 100:
        discount = self.price * (percentage / 100)
        self.price -= discount
        print(f"Applied {percentage}% discount. New price: ${self.price:.2f}")

def display_info(self):
    """Display product information"""
    print(f"Product: {self.name}")
    print(f"Price: ${self.price:.2f}")
    print(f"Quantity: {self.quantity}")
    print(f"Total Value: ${self.total_value():.2f}")
```

```
product = Product("Laptop", 999.99, 10)
product.display_info()
product.apply_discount(10)
product.update_quantity(5)
product.display_info()
```

Section B Solutions: Methods and Attributes

Problem 7: Counter class with tracking

```
class Counter:
total_counters = 0 # Class attribute

def __init__(self, initial_value=0):
    """Initialize counter"""
    self.value = initial_value
    Counter.total_counters += 1

def increment(self, amount=1):
    """Increment counter value"""
    self.value += amount

def decrement(self, amount=1):
    """Decrement counter value"""
    self.value -= amount

def reset(self):
    """Reset counter to zero"""
    self.value = 0

@classmethod
def get_total_counters(cls):
    """Return total number of counters created"""
    return cls.total_counters

counter1 = Counter()
counter2 = Counter(10)

counter1.increment(5)
counter2.decrement(3)

print(f"Counter 1: {counter1.value}")
print(f"Counter 2: {counter2.value}")
print(f"Total counters created: {Counter.get_total_counters()}")
```

Problem 8: Temperature class with properties

```
class Temperature:
def init(self, celsius):
"""Initialize temperature in Celsius"""
self._celsius = celsius
```

```
@property
def celsius(self):
    """Get temperature in Celsius"""
    return self._celsius

@celsius.setter
def celsius(self, value):
    """Set temperature in Celsius"""
    if value < -273.15:
        raise ValueError("Temperature cannot be below absolute zero (-273.15°C)")
    self._celsius = value

@property
def fahrenheit(self):
    """Get temperature in Fahrenheit"""
    return (self._celsius * 9/5) + 32

@fahrenheit.setter
def fahrenheit(self, value):
    """Set temperature using Fahrenheit"""
    celsius = (value - 32) * 5/9
    if celsius < -273.15:
        raise ValueError("Temperature cannot be below absolute zero")
    self._celsius = celsius

@property
def kelvin(self):
    """Get temperature in Kelvin"""
    return self._celsius + 273.15

@kelvin.setter
def kelvin(self, value):
    """Set temperature using Kelvin"""
    if value < 0:
        raise ValueError("Kelvin cannot be negative")
    self._celsius = value - 273.15
```

```
temp = Temperature(25)
print(f"Celsius: {temp.celsius}°C")
print(f"Fahrenheit: {temp.fahrenheit}°F")
print(f"Kelvin: {temp.kelvin}K")

temp.fahrenheit = 100
print(f"After setting to 100°F: {temp.celsius:.2f}°C")
```

Problem 9: Person class with static methods

```
import re
```

```
class Person:
def init(self, name, email, phone):
"""Initialize person with validated contact info"""
self.name = name
if Person.validate_email(email):
self.email = email
else:
raise ValueError("Invalid email address")
```

```
    if Person.validate_phone(phone):
        self.phone = phone
    else:
        raise ValueError("Invalid phone number")

@staticmethod
def validate_email(email):
    """Validate email address format"""
    pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
    return bool(re.match(pattern, email))

@staticmethod
def validate_phone(phone):
    """Validate phone number format (10 digits)"""
    pattern = r'^\d{10}$'
    return bool(re.match(pattern, phone))

def display_info(self):
    """Display person information"""
    print(f"Name: {self.name}")
    print(f"Email: {self.email}")
    print(f"Phone: {self.phone}")
```

Test validation

```
print(Person.validate_email("test@example.com")) # True
print(Person.validate_email("invalid.email")) # False
print(Person.validate_phone("1234567890")) # True
print(Person.validate_phone("123")) # False

person = Person("Alice", "alice@example.com", "1234567890")
person.display_info()
```

Problem 10: Library class with class methods

```
class Library:
total_books = 0
total_libraries = 0
```

```
def __init__(self, name, location):
    """Initialize library"""
    self.name = name
    self.location = location
    self.books = []
    Library.total_libraries += 1

def add_book(self, book_title):
    """Add a book to this library"""
    self.books.append(book_title)
    Library.total_books += 1

@classmethod
def get_total_books(cls):
    """Get total books across all libraries"""
    return cls.total_books

@classmethod
def get_total_libraries(cls):
    """Get total number of libraries"""
    return cls.total_libraries

@classmethod
def create_default_library(cls):
    """Factory method to create a default library"""
    return cls("Central Library", "Downtown")

lib1 = Library("City Library", "Main Street")
lib2 = Library("University Library", "Campus")

lib1.add_book("Python Programming")
lib1.add_book("Data Science Handbook")
lib2.add_book("Machine Learning Basics")

print(f"Total libraries: {Library.get_total_libraries()}")
print(f"Total books: {Library.get_total_books()}")

default_lib = Library.create_default_library()
print(f"Created: {default_lib.name} at {default_lib.location}")
```

Problem 11: Date class with comparison operators

```
class Date:
def init(self, day, month, year):
"""Initialize date"""
self.day = day
self.month = month
self.year = year
```

```
def to_days(self):
    """Convert date to total days for comparison"""
    return self.year * 365 + self.month * 30 + self.day

def __eq__(self, other):
    """Check if two dates are equal"""
    return self.to_days() == other.to_days()

def __lt__(self, other):
    """Check if this date is less than another"""
    return self.to_days() < other.to_days()

def __gt__(self, other):
    """Check if this date is greater than another"""
    return self.to_days() > other.to_days()

def __le__(self, other):
    """Check if this date is less than or equal to another"""
    return self.to_days() <= other.to_days()

def __ge__(self, other):
    """Check if this date is greater than or equal to another"""
    return self.to_days() >= other.to_days()

def __str__(self):
    """String representation"""
    return f"{self.day:02d}/{self.month:02d}/{self.year}"
```

```
date1 = Date(15, 6, 2024)
date2 = Date(20, 6, 2024)
date3 = Date(15, 6, 2024)

print(f"Date 1: {date1}")
print(f"Date 2: {date2}")
print(f"Date 1 == Date 3: {date1 == date3}")
print(f"Date 1 < Date 2: {date1 < date2}")
print(f"Date 2 > Date 1: {date2 > date1}")
```

Problem 12: Time class with addition

```
class Time:
def init(self, hour, minute, second):
"""Initialize time"""
self.hour = hour
self.minute = minute
self.second = second
self._normalize()
```

```
def _normalize(self):
    """Normalize time values"""
    if self.second >= 60:
        self.minute += self.second // 60
        self.second %= 60

    if self.minute >= 60:
        self.hour += self.minute // 60
        self.minute %= 60

    if self.hour >= 24:
        self.hour %= 24

def __add__(self, other):
    """Add two time objects"""
    total_seconds = (self.hour * 3600 + self.minute * 60 + self.second +
                     other.hour * 3600 + other.minute * 60 + other.second)

    hours = (total_seconds // 3600) % 24
    minutes = (total_seconds % 3600) // 60
    seconds = total_seconds % 60

    return Time(hours, minutes, seconds)

def __str__(self):
    """String representation"""
    return f"{self.hour:02d}:{self.minute:02d}:{self.second:02d}"
```

```
time1 = Time(10, 30, 45)
time2 = Time(2, 45, 30)

time3 = time1 + time2
print(f"Time 1: {time1}")
print(f"Time 2: {time2}")
print(f"Sum: {time3}")
```

Section C Solutions: Access Specifiers

Problem 13: CreditCard class with access levels

```
class CreditCard:
def init(self, card_holder, card_number, cvv, expiry):
"""Initialize credit card"""
self.card_holder = card_holder # Public
self._card_number = card_number # Protected
self.__cvv = cvv # Private
self._expiry = expiry # Protected
```

```
def get_masked_number(self):
    """Return masked card number"""
    return f"****-****-****-{self._card_number[-4:]}"

def validate_cvv(self, cvv):
    """Validate CVV (without exposing it)"""
    return self.__cvv == cvv

def display_info(self):
    """Display card information"""
    print(f"Card Holder: {self.card_holder}")
    print(f"Card Number: {self.get_masked_number()}")
    print(f"Expiry: {self._expiry}")
```

```
card = CreditCard("Alice Johnson", "1234567890123456", "123", "12/25")
card.display_info()
print(f"\nCVV validation (123): {card.validate_cvv('123')}")
print(f"CVV validation (456): {card.validate_cvv('456')}")
```

Problem 14: User class with authentication

```
import hashlib

class User:
def init(self, username, email, password):
"""Initialize user with security measures"""
self.username = username # Public
self._email = email # Protected
self.__password = self._hash_password(password) # Private
```

```
def _hash_password(self, password):
    """Hash password (protected method)"""
    return hashlib.sha256(password.encode()).hexdigest()

def validate_password(self, password):
    """Validate password without exposing it"""
    return self.__password == self._hash_password(password)

def reset_password(self, old_password, new_password):
    """Reset password with validation"""
    if self.validate_password(old_password):
        self.__password = self._hash_password(new_password)
        print("Password reset successful")
        return True
    else:
        print("Invalid old password")
        return False

def get_email(self):
```

```
    """Public method to access protected email"""
    return self._email

def update_email(self, new_email):
    """Update email with validation"""
    if "@" in new_email and "." in new_email:
        self._email = new_email
        print("Email updated successfully")
    else:
        print("Invalid email format")

user = User("alice123", "alice@example.com", "secure_password")
print(f"Username: {user.username}")
print(f"Email: {user.get_email()}")
print(f"Password valid: {user.validate_password('secure_password')}")
user.reset_password("secure_password", "new_secure_password")
```

Problem 15: Employee class demonstrating all access levels

```
class Employee:
company_name = "TechCorp" # Public class attribute

def __init__(self, name, emp_id, salary, ssn):
    """Initialize employee with various access levels"""
    self.name = name                    # Public
    self._emp_id = emp_id               # Protected
    self.__salary = salary              # Private
    self.__ssn = ssn                    # Private (sensitive)

# Public methods
def display_info(self):
    """Display public employee information"""
    print(f"Name: {self.name}")
    print(f"Employee ID: {self._emp_id}")
    print(f"Company: {Employee.company_name}")

def get_salary(self):
    """Public getter for private salary"""
    return self.__salary

# Protected methods
def _calculate_bonus(self):
    """Calculate bonus (for internal/subclass use)"""
    return self.__salary * 0.10

def _generate_report(self):
    """Generate internal report (protected)"""
    return f"Employee {self._emp_id}: Salary ${self.__salary}"
```

```
# Private methods
def __validate_ssn(self):
    """Validate SSN format (private)"""
    return len(str(self.__ssn)) == 9

def __encrypt_data(self):
    """Encrypt sensitive data (private)"""
    return "***encrypted***"
```

```
emp = Employee("John Doe", "E001", 75000, 123456789)
emp.display_info()
print(f"Salary: ${emp.get_salary()}")
print(f"Protected emp_id accessible: {emp._emp_id}")
```

print(emp.__salary) # Error - private attribute

Problem 16: Database class with security

```
class Database:
def init(self, host, username, password):
"""Initialize database connection"""
self.__host = host # Private
self.__username = username # Private
self.__password = password # Private
self.__connection = None # Private
self.is_connected = False # Public status
```

```
def __create_connection(self):
    """Create database connection (private)"""
    # Simulate connection
    self.__connection = f"Connected to {self.__host}"
    return True

def connect(self):
    """Public method to connect to database"""
    if self.__create_connection():
        self.is_connected = True
        print(f"Successfully connected to database at {self.__host}")
    else:
        print("Connection failed")

def query(self, sql):
    """Execute a query (public interface)"""
    if not self.is_connected:
        print("Not connected to database")
        return None

    if self.__validate_query(sql):
        return self.__execute_query(sql)
```

```
    else:
        print("Invalid query")
        return None

def __validate_query(self, sql):
    """Validate SQL query (private)"""
    dangerous_keywords = ['DROP', 'DELETE', 'TRUNCATE']
    return not any(keyword in sql.upper() for keyword in dangerous_keywords)

def __execute_query(self, sql):
    """Execute validated query (private)"""
    return f"Executing: {sql}"
```

```
db = Database("localhost", "admin", "secret_password")
db.connect()
result = db.query("SELECT * FROM users")
print(result)
```

Section D Solutions: Inheritance

Problem 17: Vehicle hierarchy

```
class Vehicle:
def init(self, brand, model, year):
"""Initialize vehicle"""
self.brand = brand
self.model = model
self.year = year
self.mileage = 0
```

```
def drive(self, miles):
    """Drive the vehicle"""
    self.mileage += miles
    print(f"Drove {miles} miles. Total mileage: {self.mileage}")

def display_info(self):
    """Display vehicle information"""
    print(f"{self.year} {self.brand} {self.model}")
    print(f"Mileage: {self.mileage} miles")
```

```
class Car(Vehicle):
def init(self, brand, model, year, doors, fuel_type):
"""Initialize car"""
super().init(brand, model, year)
self.doors = doors
self.fuel_type = fuel_type
```

```
def display_info(self):
    """Display car information"""
```

```
    super().display_info()
    print(f"Doors: {self.doors}")
    print(f"Fuel Type: {self.fuel_type}")

class Motorcycle(Vehicle):
def init(self, brand, model, year, engine_cc):
"""Initialize motorcycle"""
super().init(brand, model, year)
self.engine_cc = engine_cc

def wheelie(self):
    """Perform a wheelie"""
    print(f"{self.brand} {self.model} is doing a wheelie!")

def display_info(self):
    """Display motorcycle information"""
    super().display_info()
    print(f"Engine: {self.engine_cc}cc")

car = Car("Toyota", "Camry", 2024, 4, "Hybrid")
motorcycle = Motorcycle("Harley-Davidson", "Street 750", 2024, 750)

car.drive(100)
car.display_info()

print("\n")

motorcycle.drive(50)
motorcycle.wheelie()
motorcycle.display_info()
```

Problem 18: Shape hierarchy with abstract methods

```
class Shape:
def init(self, name):
"""Initialize shape"""
self.name = name

def area(self):
    """Calculate area (must be overridden)"""
    raise NotImplementedError("Subclass must implement area()")

def perimeter(self):
    """Calculate perimeter (must be overridden)"""
    raise NotImplementedError("Subclass must implement perimeter()")

def describe(self):
    """Describe the shape"""
    print(f"This is a {self.name}")
```

```
    print(f"Area: {self.area():.2f}")
    print(f"Perimeter: {self.perimeter():.2f}")

class Rectangle(Shape):
def init(self, width, height):
"""Initialize rectangle"""
super().init("Rectangle")
self.width = width
self.height = height

def area(self):
    """Calculate rectangle area"""
    return self.width * self.height

def perimeter(self):
    """Calculate rectangle perimeter"""
    return 2 * (self.width + self.height)

class Circle(Shape):
def init(self, radius):
"""Initialize circle"""
super().init("Circle")
self.radius = radius

def area(self):
    """Calculate circle area"""
    return 3.14159 * self.radius ** 2

def perimeter(self):
    """Calculate circle circumference"""
    return 2 * 3.14159 * self.radius

class Triangle(Shape):
def init(self, side_a, side_b, side_c):
"""Initialize triangle"""
super().init("Triangle")
self.side_a = side_a
self.side_b = side_b
self.side_c = side_c

def area(self):
    """Calculate triangle area using Heron's formula"""
    s = self.perimeter() / 2
    return (s * (s - self.side_a) * (s - self.side_b) * (s - self.side_c)) ** 0.5

def perimeter(self):
    """Calculate triangle perimeter"""
```

```
    return self.side_a + self.side_b + self.side_c
```

Create and describe shapes

```
shapes = [
Rectangle(5, 3),
Circle(4),
Triangle(3, 4, 5)
]

for shape in shapes:
shape.describe()
print()
```

Problem 19: Animal hierarchy with polymorphism

```
class Animal:
def init(self, name, age):
"""Initialize animal"""
self.name = name
self.age = age

def make_sound(self):
    """Make a sound (to be overridden)"""
    print(f"{self.name} makes a sound")

def eat(self):
    """Eating behavior"""
    print(f"{self.name} is eating")

def info(self):
    """Display animal information"""
    print(f"Name: {self.name}, Age: {self.age} years")

class Dog(Animal):
def init(self, name, age, breed):
"""Initialize dog"""
super().init(name, age)
self.breed = breed

def make_sound(self):
    """Dog barks"""
    print(f"{self.name} barks: Woof! Woof!")

def fetch(self):
    """Dog-specific behavior"""
    print(f"{self.name} is fetching the ball")
```

```
class Cat(Animal):
def init(self, name, age, color):
"""Initialize cat"""
super().init(name, age)
self.color = color

def make_sound(self):
    """Cat meows"""
    print(f"{self.name} meows: Meow! Meow!")

def scratch(self):
    """Cat-specific behavior"""
    print(f"{self.name} is scratching the furniture")

class Bird(Animal):
def init(self, name, age, species):
"""Initialize bird"""
super().init(name, age)
self.species = species

def make_sound(self):
    """Bird chirps"""
    print(f"{self.name} chirps: Tweet! Tweet!")

def fly(self):
    """Bird-specific behavior"""
    print(f"{self.name} is flying")
```

Demonstrate polymorphism

```
animals = [
Dog("Buddy", 4, "Golden Retriever"),
Cat("Whiskers", 3, "Orange"),
Bird("Tweety", 2, "Canary")
]

for animal in animals:
animal.info()
animal.make_sound()
animal.eat()
print()
```

Problem 20: Person, Student, Teacher hierarchy

```
class Person:
def init(self, name, age, address):
"""Initialize person"""
self.name = name
```

```
self.age = age
self.address = address

def introduce(self):
    """Introduce the person"""
    print(f"Hi, I'm {self.name} and I'm {self.age} years old")

def display_info(self):
    """Display person information"""
    print(f"Name: {self.name}")
    print(f"Age: {self.age}")
    print(f"Address: {self.address}")

class Student(Person):
def init(self, name, age, address, student_id, major):
"""Initialize student"""
super().init(name, age, address)
self.student_id = student_id
self.major = major
self.grades = {}

def add_grade(self, course, grade):
    """Add a course grade"""
    self.grades[course] = grade

def calculate_gpa(self):
    """Calculate GPA"""
    if not self.grades:
        return 0.0
    return sum(self.grades.values()) / len(self.grades) / 25  # Assuming 100-point scale

def introduce(self):
    """Student introduction"""
    super().introduce()
    print(f"I'm a {self.major} student with ID {self.student_id}")

def display_info(self):
    """Display student information"""
    super().display_info()
    print(f"Student ID: {self.student_id}")
    print(f"Major: {self.major}")
    print(f"GPA: {self.calculate_gpa():.2f}")

class Teacher(Person):
def init(self, name, age, address, employee_id, subject, salary):
"""Initialize teacher"""
super().init(name, age, address)
```

```
self.employee_id = employee_id
self.subject = subject
self.salary = salary
self.courses = []
```

```
def assign_course(self, course):
    """Assign a course to teacher"""
    self.courses.append(course)
    print(f"{self.name} has been assigned to teach {course}")

def introduce(self):
    """Teacher introduction"""
    super().introduce()
    print(f"I'm a {self.subject} teacher with ID {self.employee_id}")

def display_info(self):
    """Display teacher information"""
    super().display_info()
    print(f"Employee ID: {self.employee_id}")
    print(f"Subject: {self.subject}")
    print(f"Courses: {', '.join(self.courses) if self.courses else 'None'}")
```

```
student = Student("Alice", 20, "123 College St", "S12345", "Computer Science")
student.add_grade("Python Programming", 95)
student.add_grade("Data Structures", 90)
student.introduce()
student.display_info()

print("\n")

teacher = Teacher("Dr. Smith", 45, "456 Faculty Ave", "T001", "Mathematics", 75000)
teacher.assign_course("Calculus I")
teacher.assign_course("Linear Algebra")
teacher.introduce()
teacher.display_info()
```

Section E Solutions: Polymorphism

Problem 25: Payment system with polymorphism

```
class PaymentMethod:
def process_payment(self, amount):
"""Process payment (to be overridden)"""
raise NotImplementedError("Subclass must implement process_payment()")
```

```
def validate(self):
    """Validate payment method"""
    raise NotImplementedError("Subclass must implement validate()")
```

```
class CreditCard(PaymentMethod):
def init(self, card_number, card_holder, cvv):
"""Initialize credit card"""
self.card_number = card_number
self.card_holder = card_holder
self.cvv = cvv

def validate(self):
    """Validate credit card"""
    return len(self.card_number) == 16 and len(self.cvv) == 3

def process_payment(self, amount):
    """Process credit card payment"""
    if self.validate():
        print(f"Processing ${amount} via Credit Card")
        print(f"Card ending in {self.card_number[-4:]}")
        print("Payment successful!")
        return True
    else:
        print("Invalid credit card")
        return False
```

```
class DebitCard(PaymentMethod):
def init(self, card_number, pin):
"""Initialize debit card"""
self.card_number = card_number
self.pin = pin

def validate(self):
    """Validate debit card"""
    return len(self.card_number) == 16 and len(self.pin) == 4

def process_payment(self, amount):
    """Process debit card payment"""
    if self.validate():
        print(f"Processing ${amount} via Debit Card")
        print(f"Card ending in {self.card_number[-4:]}")
        print("Payment successful!")
        return True
    else:
        print("Invalid debit card")
        return False
```

```
class UPI(PaymentMethod):
def init(self, upi_id):
"""Initialize UPI"""
self.upi_id = upi_id
```

```
def validate(self):
    """Validate UPI ID"""
    return "@" in self.upi_id

def process_payment(self, amount):
    """Process UPI payment"""
    if self.validate():
        print(f"Processing ${amount} via UPI")
        print(f"UPI ID: {self.upi_id}")
        print("Payment successful!")
        return True
    else:
        print("Invalid UPI ID")
        return False
```

```
def checkout(payment_method, amount):
"""Process checkout with any payment method"""
print(f"\n--- Processing Payment of ${amount} ---")
payment_method.process_payment(amount)
print("--- Transaction Complete ---\n")
```

Demonstrate polymorphism

```
credit_card = CreditCard("1234567890123456", "John Doe", "123")
debit_card = DebitCard("9876543210987654", "4567")
upi = UPI("user@bank")

checkout(credit_card, 100.50)
checkout(debit_card, 75.25)
checkout(upi, 200.00)
```

Problem 28: Fraction class with operator overloading

```
import math

class Fraction:
def init(self, numerator, denominator):
"""Initialize fraction"""
if denominator == 0:
raise ValueError("Denominator cannot be zero")
```

```
    # Reduce fraction
    gcd = math.gcd(numerator, denominator)
    self.numerator = numerator // gcd
    self.denominator = denominator // gcd

    # Handle negative fractions
    if self.denominator < 0:
        self.numerator = -self.numerator
        self.denominator = -self.denominator
```

```
def __add__(self, other):
    """Add two fractions"""
    num = self.numerator * other.denominator + other.numerator * self.denominator
    denom = self.denominator * other.denominator
    return Fraction(num, denom)

def __sub__(self, other):
    """Subtract two fractions"""
    num = self.numerator * other.denominator - other.numerator * self.denominator
    denom = self.denominator * other.denominator
    return Fraction(num, denom)

def __mul__(self, other):
    """Multiply two fractions"""
    num = self.numerator * other.numerator
    denom = self.denominator * other.denominator
    return Fraction(num, denom)

def __truediv__(self, other):
    """Divide two fractions"""
    num = self.numerator * other.denominator
    denom = self.denominator * other.numerator
    return Fraction(num, denom)

def __eq__(self, other):
    """Check equality"""
    return (self.numerator == other.numerator and
            self.denominator == other.denominator)

def __str__(self):
    """String representation"""
    if self.denominator == 1:
        return str(self.numerator)
    return f"{self.numerator}/{self.denominator}"

def __repr__(self):
    """Developer representation"""
    return f"Fraction({self.numerator}, {self.denominator})"

def to_decimal(self):
    """Convert to decimal"""
    return self.numerator / self.denominator
```

Demonstrate operator overloading

```
f1 = Fraction(1, 2)
f2 = Fraction(1, 3)

print(f"f1 = {f1}")
print(f"f2 = {f2}")
print(f"f1 + f2 = {f1 + f2}")
print(f"f1 - f2 = {f1 - f2}")
print(f"f1 * f2 = {f1 * f2}")
print(f"f1 / f2 = {f1 / f2}")
print(f"f1 == f2 = {f1 == f2}")
print(f"f1 as decimal = {f1.to_decimal()}")
```

Summary

This comprehensive guide covered Python's Object-Oriented Programming fundamentals including classes, objects, methods, attributes, access specifiers, inheritance, and polymorphism. OOP provides a powerful paradigm for organizing code around real-world entities with data and behavior bundled together.

Key concepts include understanding class blueprints and object instances, using instance, class, and static methods appropriately, implementing access control through naming conventions, leveraging inheritance for code reuse and hierarchical relationships, and applying polymorphism for flexible interfaces. Mastering these OOP principles enables writing more modular, maintainable, and scalable Python applications that model complex systems effectively.

The practice problems and solutions provide hands-on experience with all major OOP concepts, from basic class creation to advanced design patterns, helping solidify understanding through practical application.